



Software Engineering

LECTURE 11: SOFTWARE ARCHITECTURE

Software architecture

Software architecture refers to the

- **high level structures** of a software system
- **discipline** of creating such structures,
- and the **documentation** of these structures.

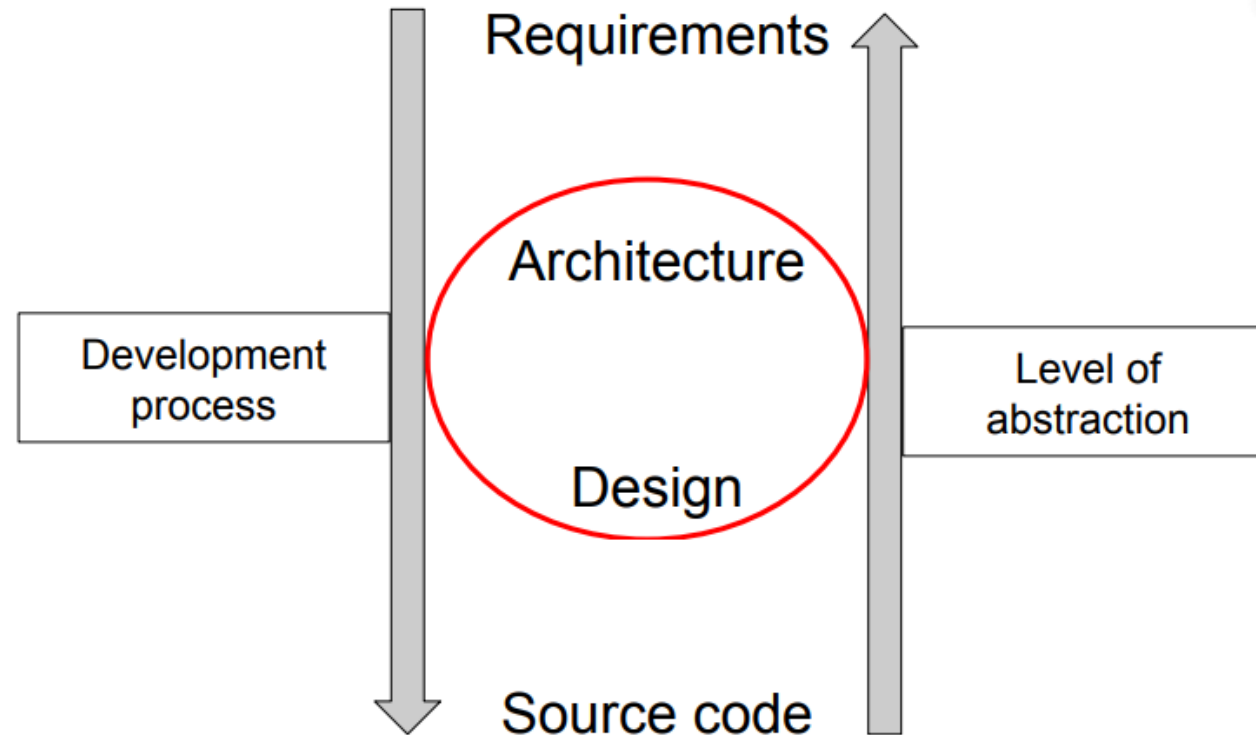
These structures are needed to reason about the software system.

Software Architecture

A software architecture defines:

- the components of the software system
- how the components use each other's functionality and data
- How control is managed between the components

Architecture vs. design



Analysis vs. Design

Analysis

Asks “what is the problem?”

- what happens in the current system?
- what is required in the new system?

Results in a detailed understanding of:

- Requirements
- Domain Properties

Focuses on the way human activities are conducted

Analysis vs. Design

Design

Investigates “how to build a solution”

- How will the new system work?
- How can we solve the problem that the analysis identified?

Results in a solution to the problem

- A working system that satisfies the requirements
- Hardware + Software + Peopleware

Focuses on building technical solutions

Separate activities, but not necessarily sequential

- ...and attempting a design usually improves understanding of the problem

Levels of abstraction

- Requirements
 - High level "what"
 - Architecture
 - High-level "how"
 - Mid-level "what"
 - Program design
 - Mid-level "how"
 - Low-level "what"
 - Code
 - Low-level "how"
- Documentation for each step should respect its level of abstraction
 - Avoid biasing later steps
 - Avoid redundancy

Architecture vs. design

Architecture (what is developed?)

High-level view of the overall system:

- What components do exist?
- What are the protocols between components?
- What type of storage etc.?

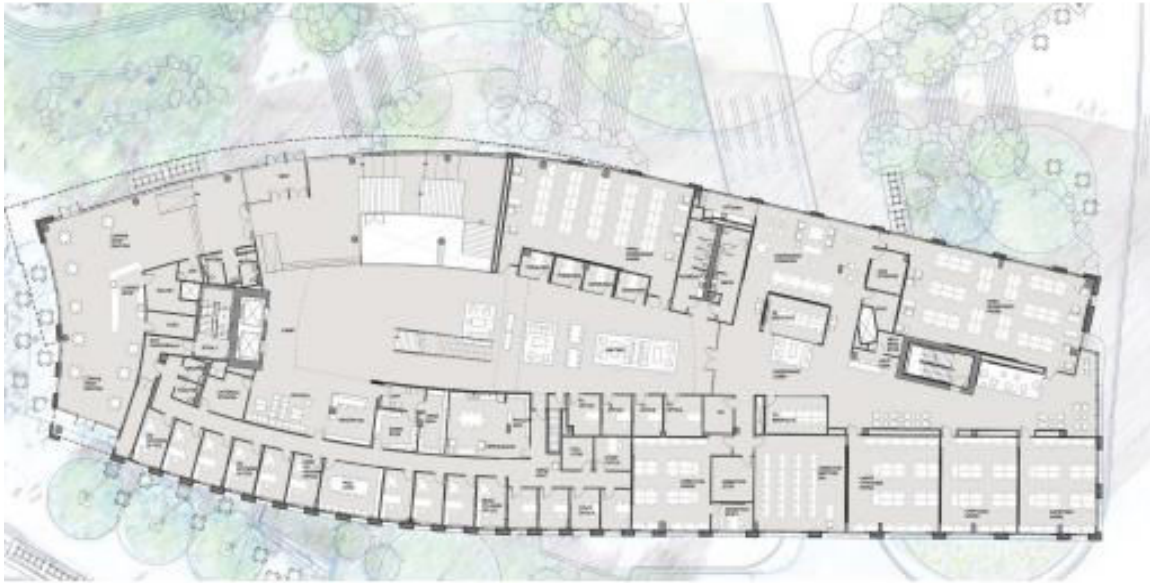
Design (how are the components developed?)

Considers individual components:

- Data representation
- Interfaces, Class hierarchy

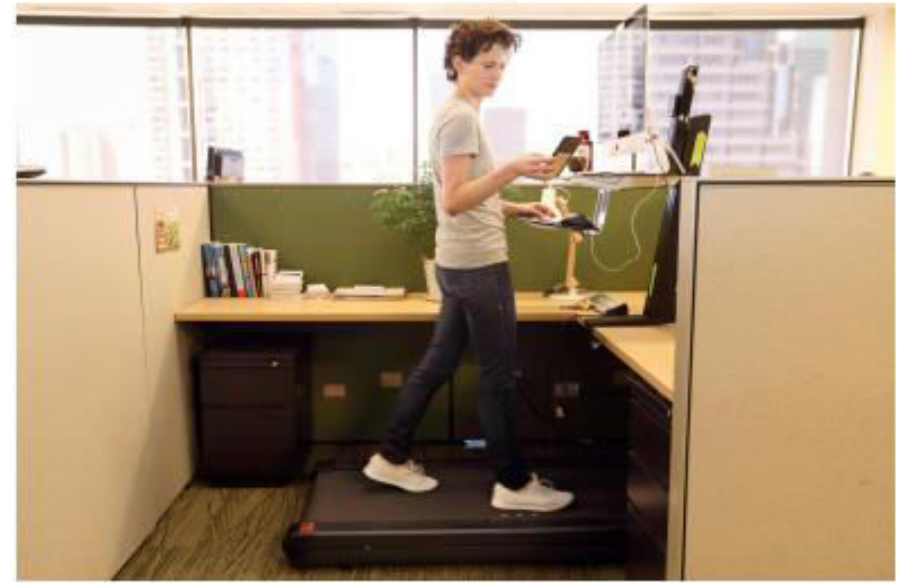
Architecture vs. design

Architecture



[Gates Center Architecture, LMN]

Design



[Office design, New York Times]

Example

- Requirements:
 - Drone can hover
- Architecture
 - Sensing → navigation → control → actuation
 - Radio input
- Program design
 - PID controller, low-pass filter
 - Gain registry
- Code
 - ```
def lpf(x1, y0, a):
 """exp filter w/
 smoothing factor a"""
 return a*x1 + (1-a)*y0
```

# Architectural considerations

---

- Infrastructure
  - Hardware
  - Operating systems
  - Virtualization
- Interfaces
  - Networks/buses
  - Protocols
- Services
  - Databases
  - Authentication
- Operations
  - Testing
  - Logging/monitoring
  - Backups
  - Rolling deployment
- Product line

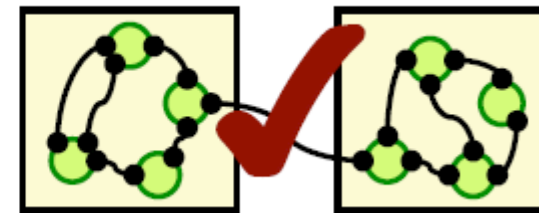
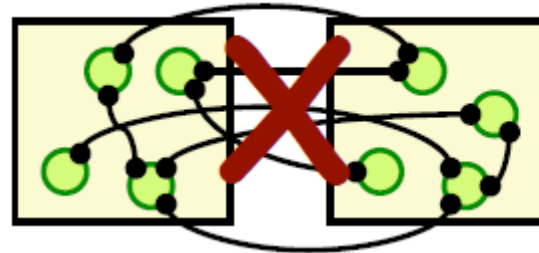
# Architectural Building blocks

Architectural Building blocks:



A good architecture:

- Minimizes coupling between modules:
  - Goal: modules don't need to know much about one another to interact
  - Low coupling makes future change easier
- Maximizes the cohesion of each module
  - Goal: the contents of each module are strongly inter-related
  - High cohesion m

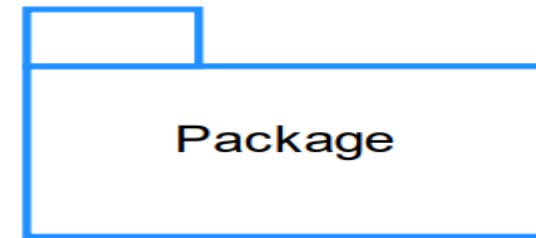


Coupling and Cohesion

# Package

General grouping of system elements

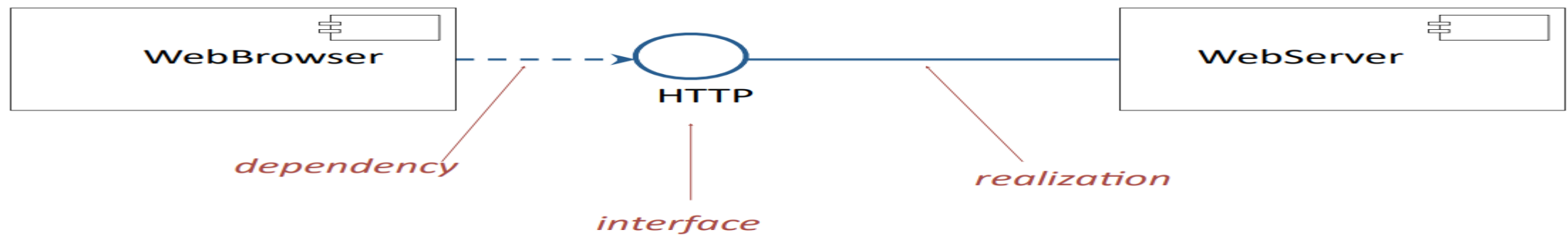
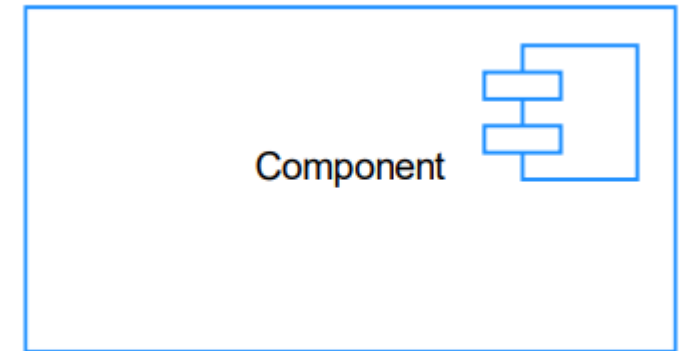
Appropriate for denoting subsystem at conceptual level



Conceptual diagram

# Component

- Replaceable part of a system
- Conforms to and realizes a set of interfaces
  - An implementation of a subsystem
  - Could be replaced by another component that realizes the same interfaces, and system would still function
- Components realize interfaces, are assembled into systems

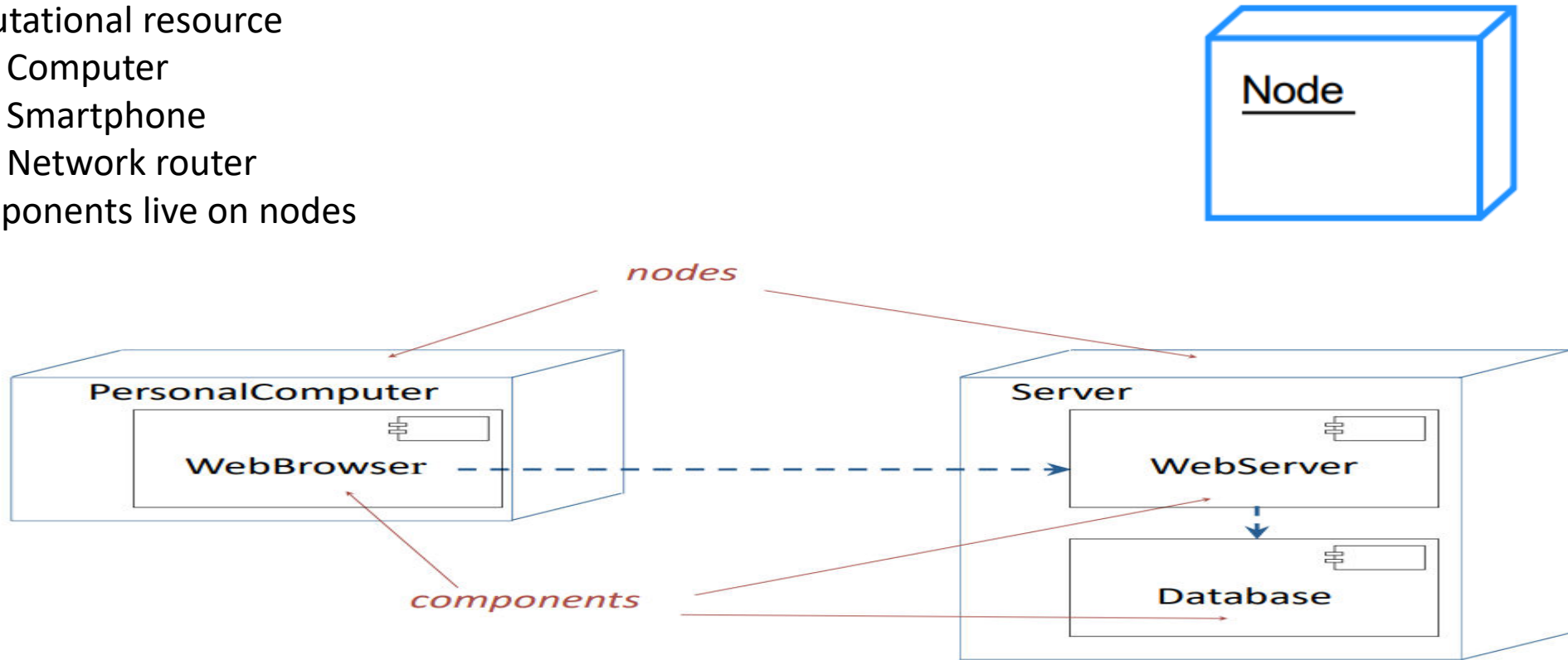


interface diagram

# Node

Physical element that exists at runtime, provides a computational resource

- Computer
- Smartphone
- Network router
- Components live on nodes



deployment diagram

# Framework

A framework, or software framework, is a platform that provides a foundation for developing software applications. Think of it as a template of a working program that can be selectively modified by adding code.

A framework is a partially complete software (sub-) system that is intended to be instantiated.

It defines the architecture for a family of (sub-) systems and provides the basic building blocks to create them.

- Web app framework.
  - React, Angular, Vue , Spring boot, Asp.net Core, Django, Rails
- Mobile app framework.
  - Flutter React Native
- Technology framework
  - **. Selenium**
- Enterprise architecture framework.
  - ITIL,IT4IT,COBIT,ISO20000,Six Sigma
  - [The Zachman Framework](#), Open Agile Architecture ([O-AA](#)), The Open Group Architectural Framework ([TOGAF](#)),
- Database framework.
  - Hadoop by Apache that stores and distributes large data sets across several servers
- Testing framework.
  - **Selenium , WebdriverIO , Cypress**
- AI application development. TensorFlow , PyTorch
- Cross-platform native apps - . [Xamarin](#)



# Architectural Patterns

---



The fundamental problem to be solved with a large system is how to break it into chunks manageable for human programmers to understand, implement, and maintain.



*An **architectural pattern** is a general, reusable solution to a commonly occurring problem in software architecture within a given context. Architectural patterns are similar to software design pattern but have a broader scope.*

# Architectural Pattern Vs Design Pattern

An architectural Pattern expresses a fundamental structural organization schema for software systems. It provides a set of predefined subsystems, their responsibilities, and includes rules and guidelines for organizing the relationships between them.

A design pattern provides a scheme for refining the subsystems or components of a software system, or the relationships between them. It describes a commonly recurring structure of communicating components that solves a general design problem within a particular context.

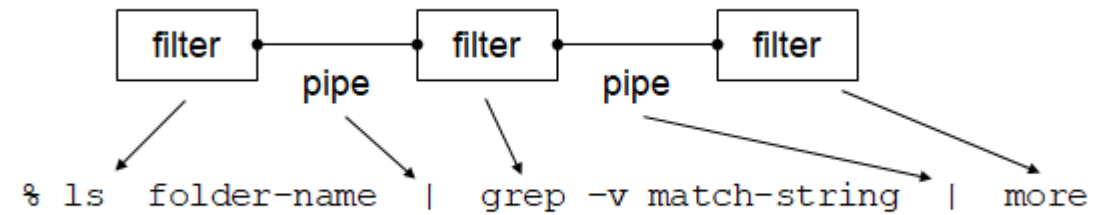
# Software Architectures

- Architectural Styles
  - Pipe and filter
  - Object oriented:
    - Client-Server; Object Broker
  - Event based
  - Layered:
    - Designing Layered Architectures
  - Repositories:
    - Blackboard, MVC

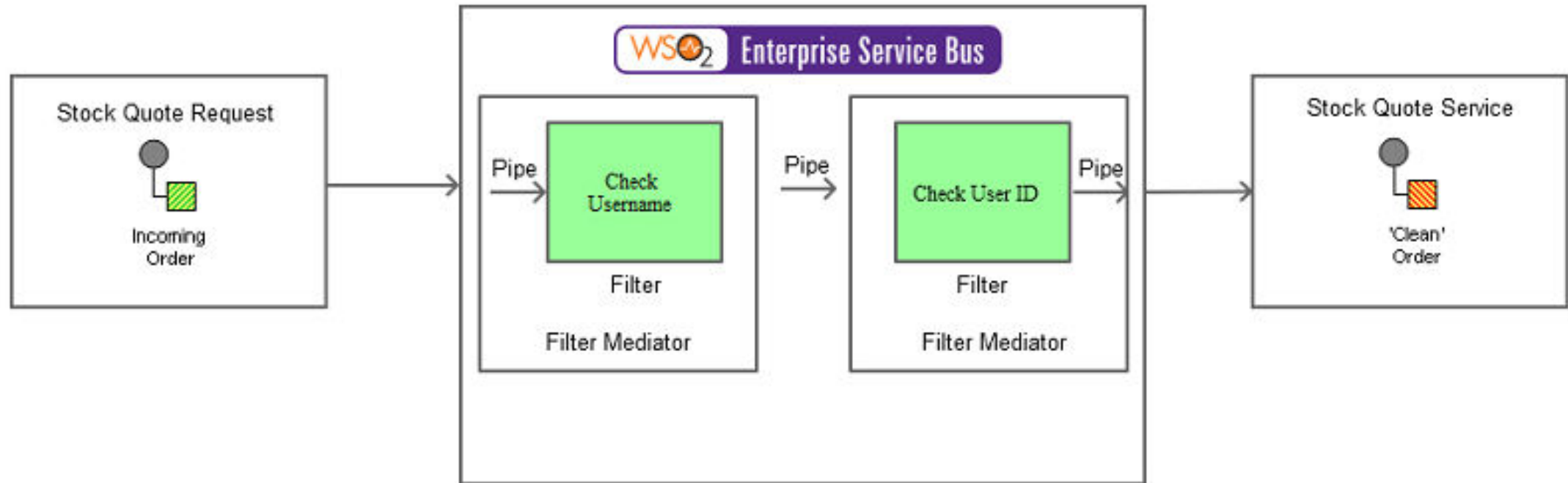


# Pipe-and-filter – Example

---



# Pipe-and-filter – Example



## Pipe and Filter Advantages

- Systems can be easily maintained and enhanced, since new filters can be added to existing systems and old filters can be replaced by improved ones.
- They permit certain kinds of specialized analysis, such as throughput and deadlock analysis.
- They naturally support concurrent execution

## Pipe and Filter Disadvantages

- Not good choice for interactive systems, because of their transformational character.

- Excessive parsing and unparsing leads to loss of performance and increased complexity in writing the filters themselves



# Object Oriented Architectures

## Examples:

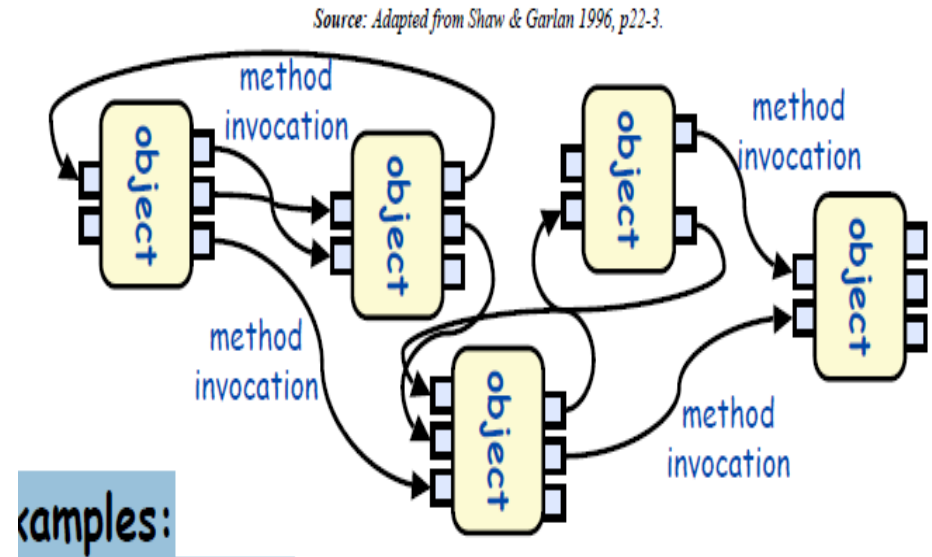
- abstract data types

## Interesting properties

- data hiding (internal data representations are not visible to clients)
- can decompose problems into sets of interacting agents
- can be multi-threaded or single thread

## Disadvantages

- objects must know the identity of object they wish to interact with



# Variant 1: Client Server

This pattern consists of two parties;

- a **server** and
- multiple **clients**.

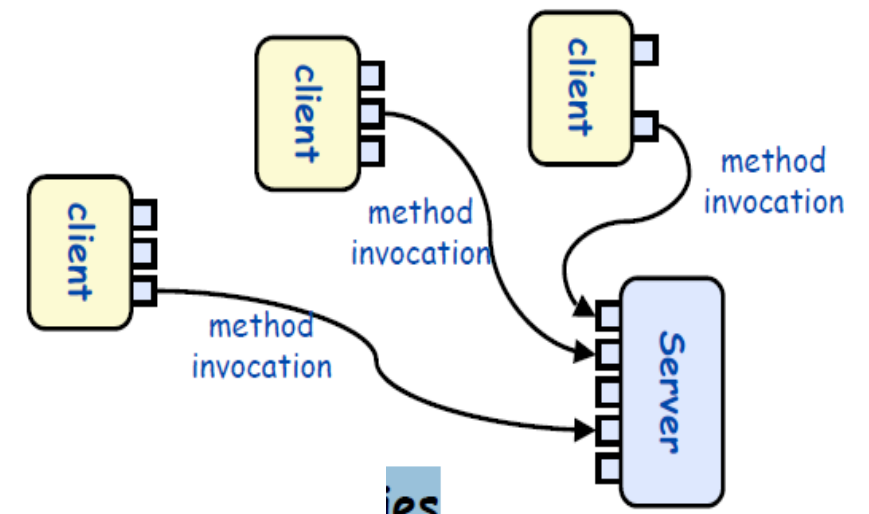
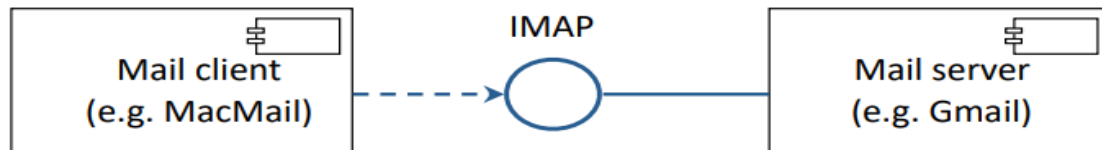
The server component will provide services to multiple client components.

Clients request services from the server and the server provides relevant services to those clients.

The server continues to listen to client requests.

## Usage

Online applications such as email, document sharing and banking.



# Variant 2- Master-slave pattern

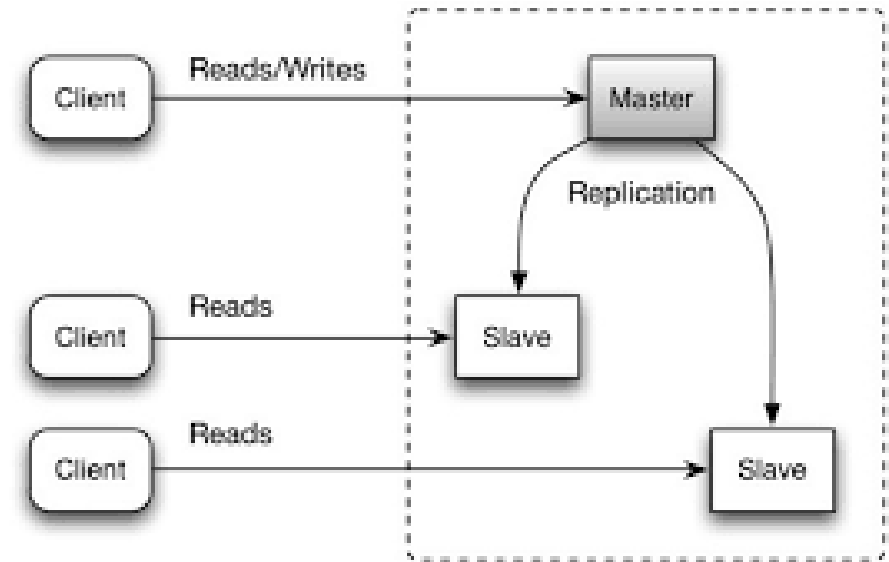
This pattern consists of two parties;

- **master**
- **slaves.**

The master component distributes the work among identical slave components, and computes a final result from the results which the slaves return.

## Usage

- In database replication, the master database is regarded as the authoritative source, and the slave databases are synchronized to it.
- Peripherals connected to a bus in a computer system (master and slave drives).



### Interesting properties

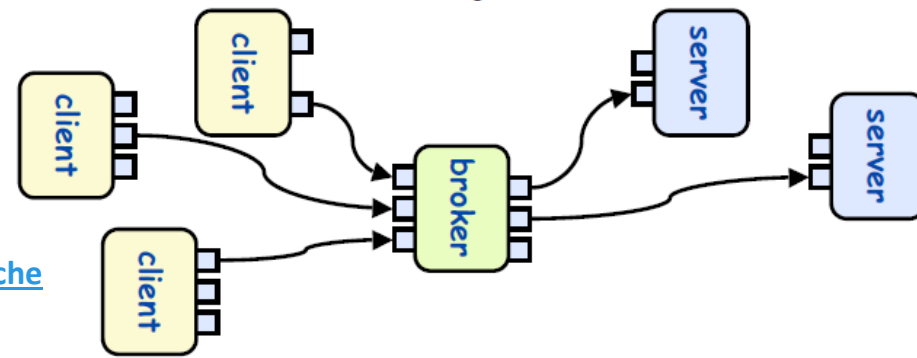
- Adds a broker between the clients and servers
- Clients no longer need to know which server they are using
- Can have many brokers, many servers.

### Disadvantages

- Broker can become a bottleneck
- Degraded performance

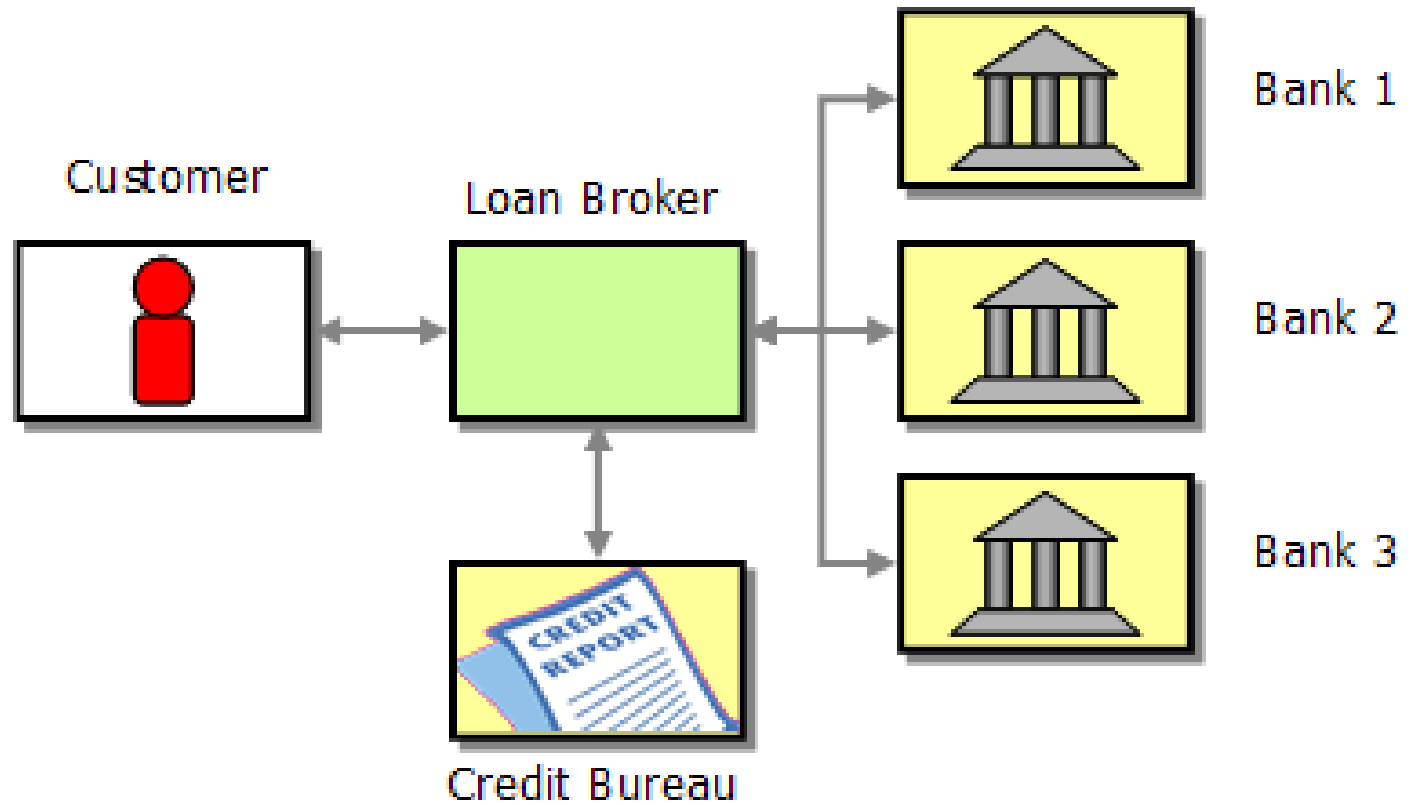
### Usage

Message broker software such as [Apache ActiveMQ](#), [Apache Kafka](#), [RabbitMQ](#) and [JBoss Messaging](#).



## Variant 3: Object Brokers

# Broker Architecture Example



# Peer-to-peer pattern

In this pattern, individual components are known as **peers**.

Peers may function both as a **client**, requesting services from other peers, and as a **server**, providing services to other peers.

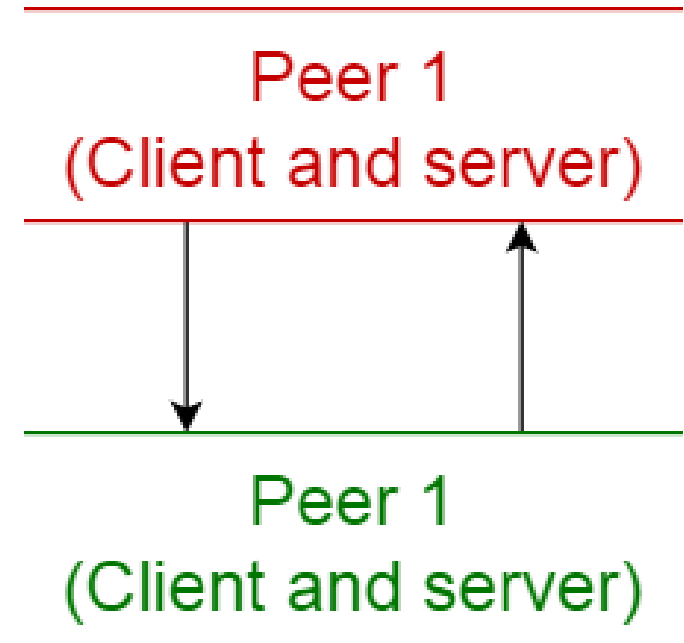
A peer may act as a client or as a server or as both, and it can change its role dynamically with time.

## Usage

File-sharing networks such as [Gnutella](#) and [G2](#))

Multimedia protocols such as [P2PTV](#) and [PDTP](#).

Proprietary multimedia applications such as [Spotify](#).



# Event based (implicit invocation)

## Examples

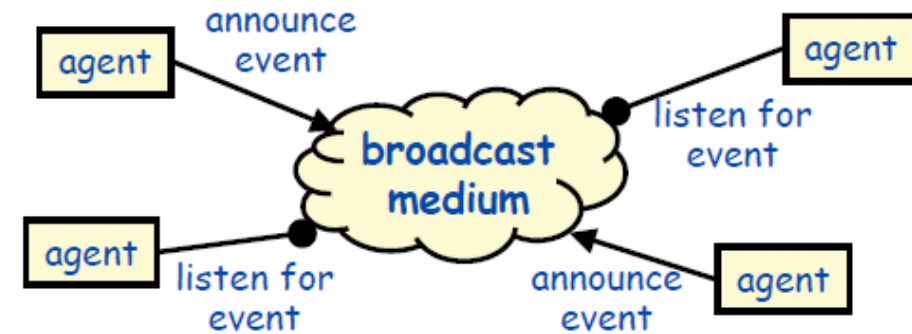
- ⇒ debugging systems (listen for particular breakpoints)
- ⇒ database management systems (for data integrity checking)
- ⇒ graphical user interfaces

## Interesting properties

- ⇒ announcers of events don't need to know who will handle the event
- ⇒ Supports re-use, and evolution of systems (add new agents easily)

## Disadvantages

- ⇒ Components have no control over ordering of computations



# Event-bus pattern

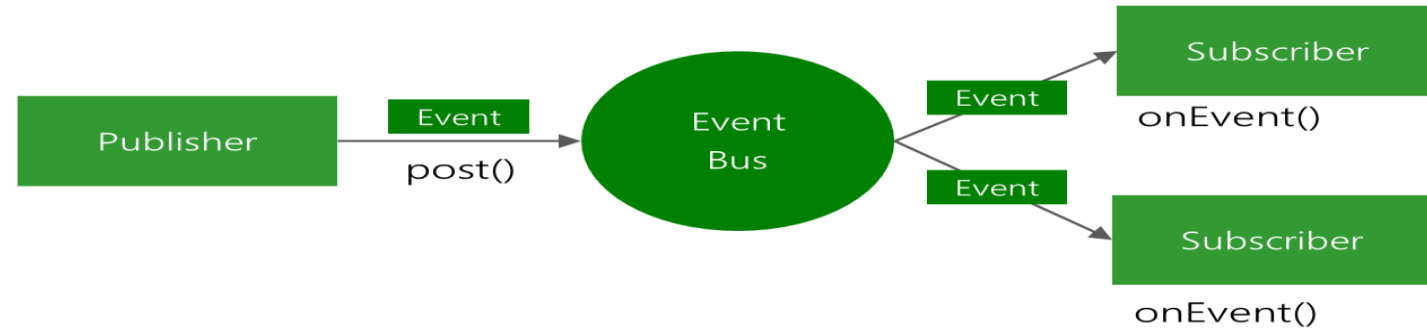
This pattern primarily deals with events and has 4 major components;

**event source,**  
**event listener,**  
**channel**  
**event bus.**

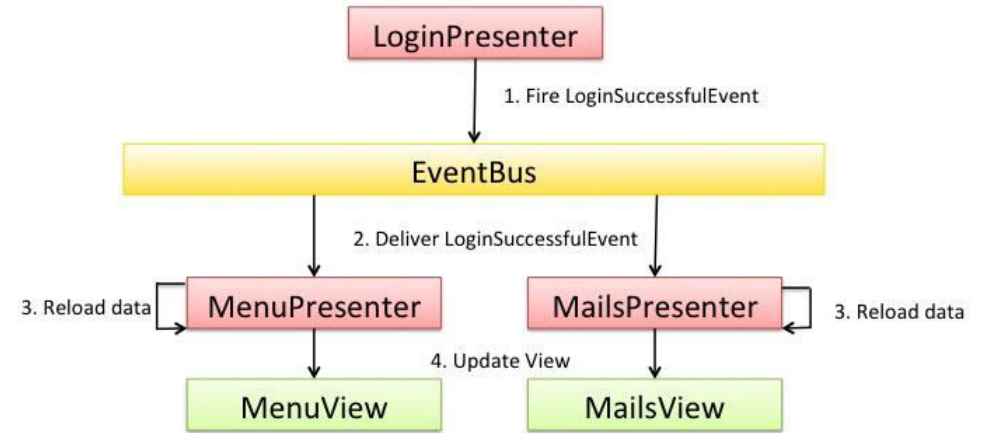
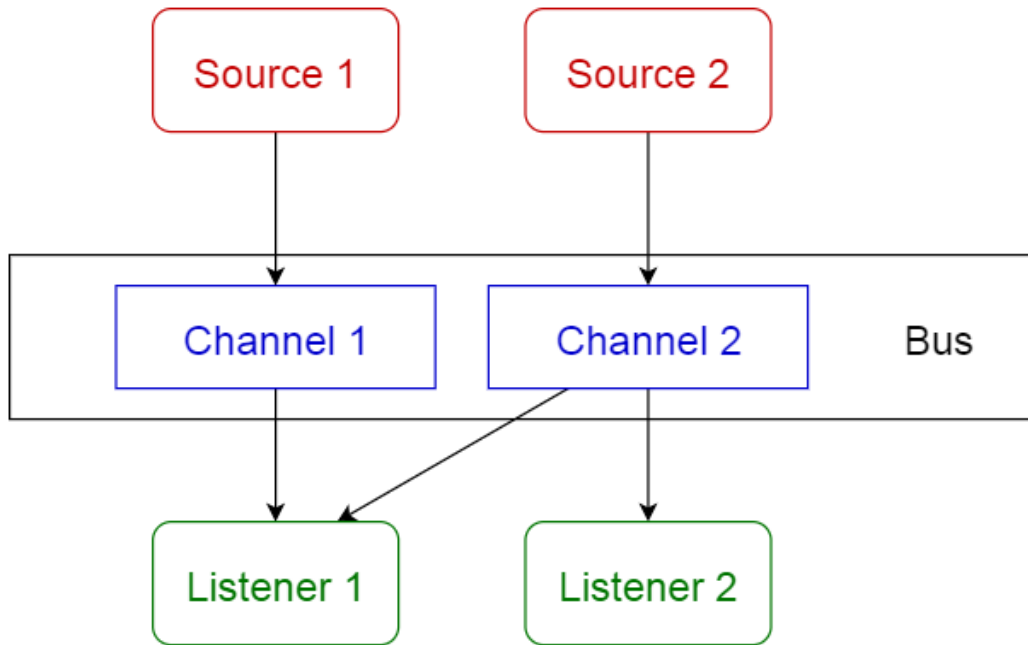
- Sources publish messages to particular channels on an event bus.
- Listeners subscribe to particular channels.
- Listeners are notified of messages that are published to a channel to which they have subscribed before.

## Usage

- Android development
- Notification services
- debugging systems (listen for particular breakpoints)
- database management systems (for data integrity checking)
- graphical user interfaces







## Event-bus pattern-Example

# Layered pattern

---

This pattern is also known as **n-tier architecture pattern**.

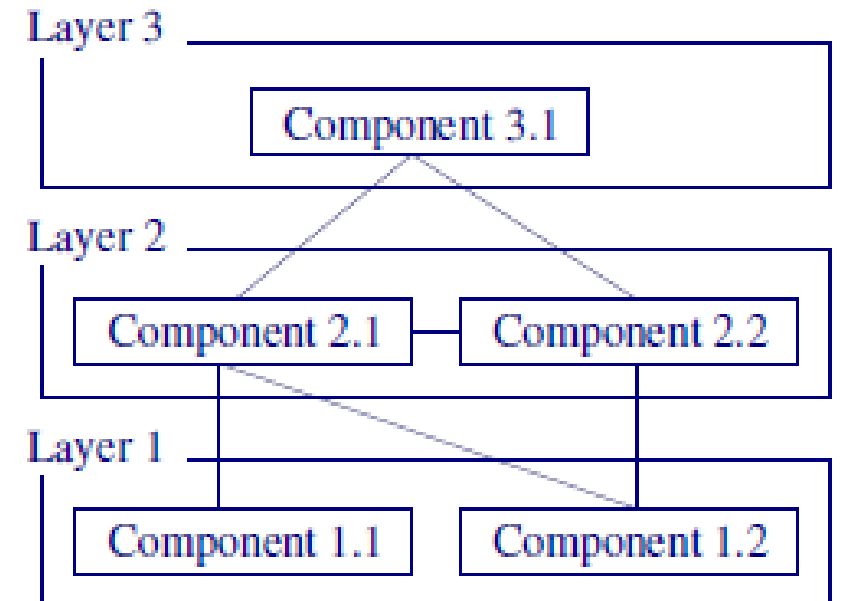
It can be used to structure programs that can be decomposed into groups of subtasks, each of which is at a particular level of abstraction.

Each layer provides services to the next higher layer.

## Usage

General desktop applications.

E commerce web applications.



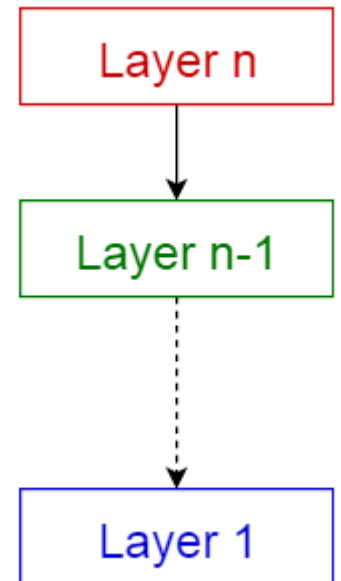
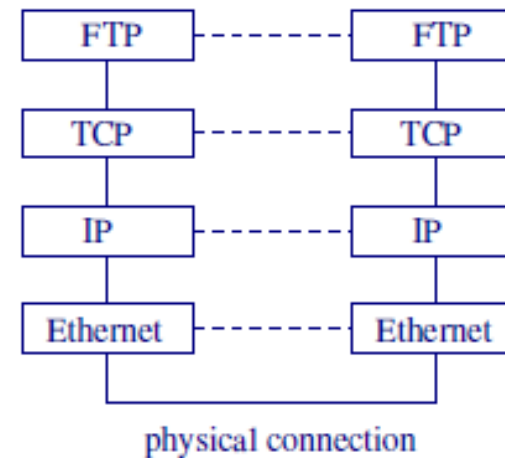
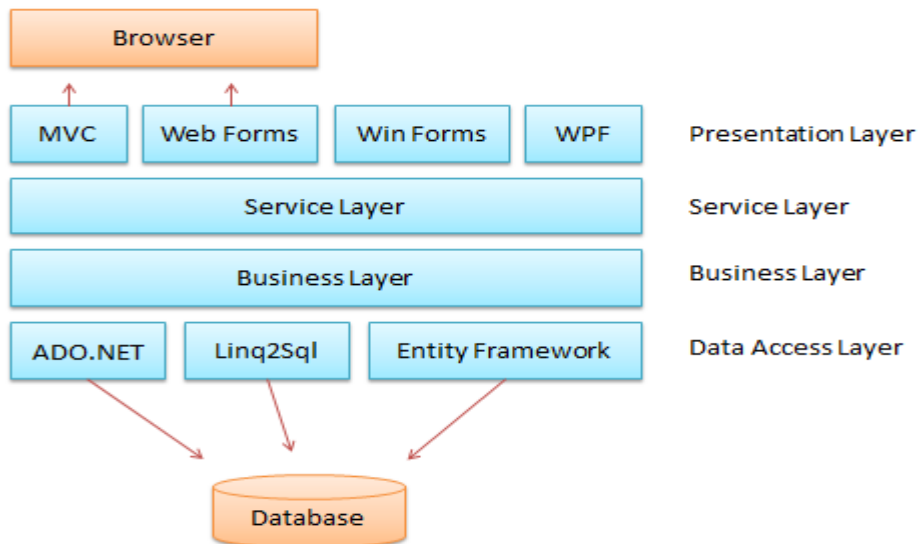
# Common layers of a general information system

**Presentation layer** (also known as **UI layer**)

**Application layer** (also known as **service layer**)

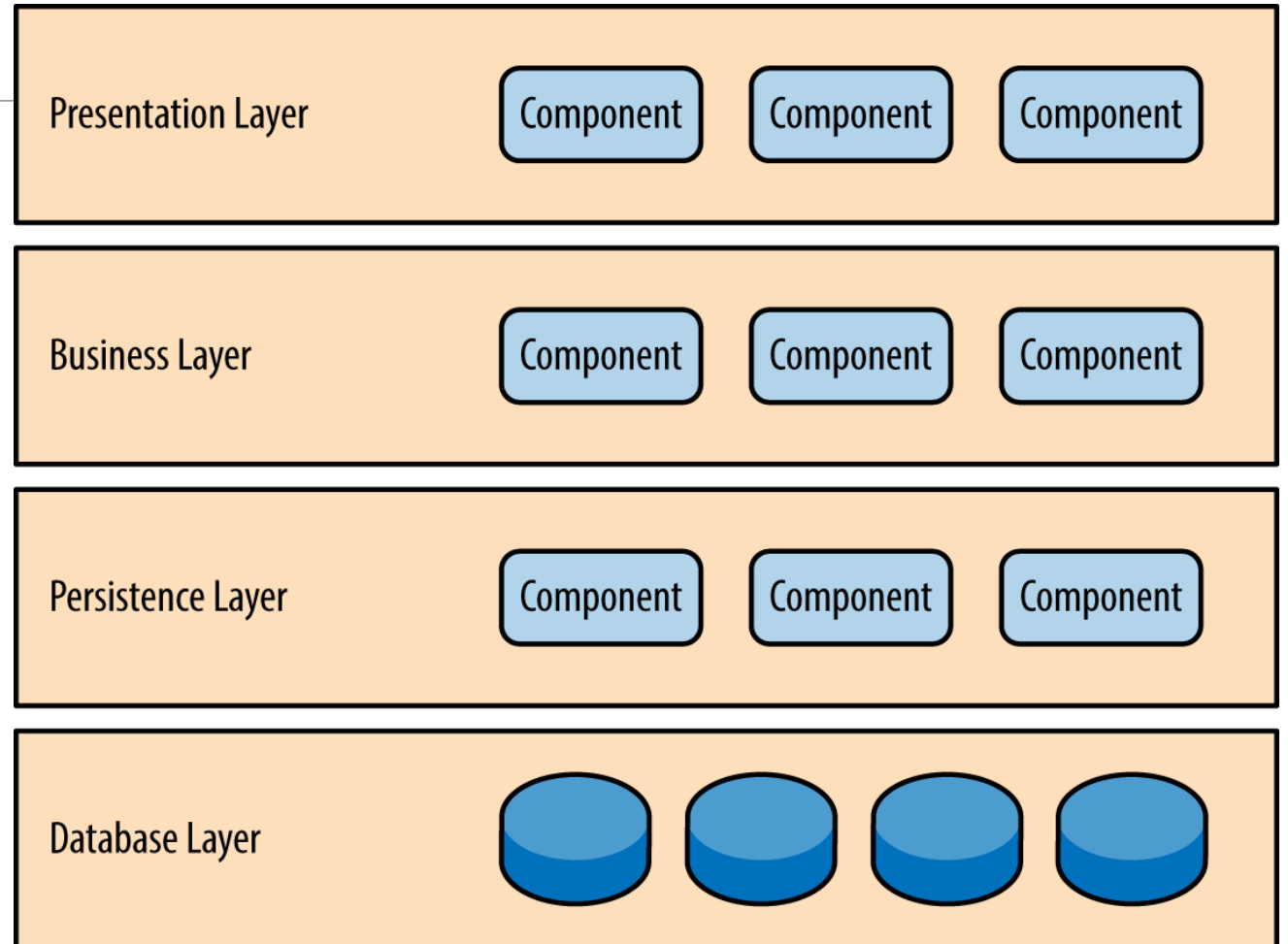
**Business logic layer** (also known as **domain layer**)

**Data access layer** (also known as **persistence layer**)

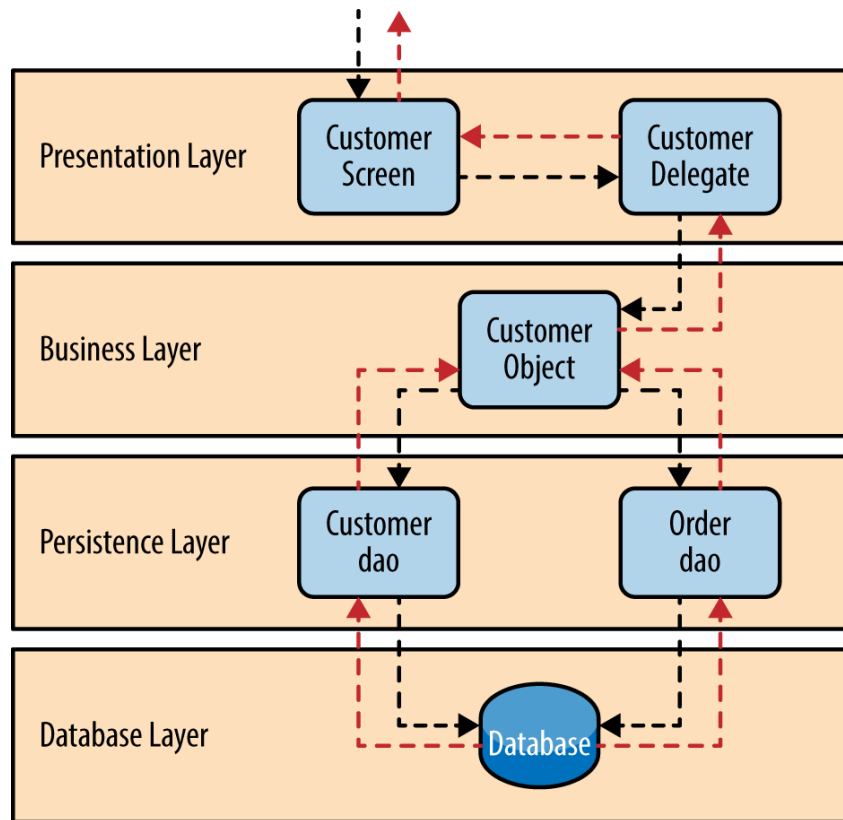


# Layered pattern

- Different levels of abstraction
- Requests go down, notifications go back up
- Possible to define stable interfaces between layers
- May add or change layers over time



# Layered pattern



Start with the lowest later

Build layers on top

Use same level of abstraction within each layer

Componenets cannot spread over two layers

Keey lower layers lean

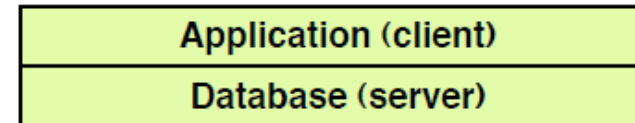
Specify interfaces for each layer

Handle errors at the lowest level possible

# How many layers?

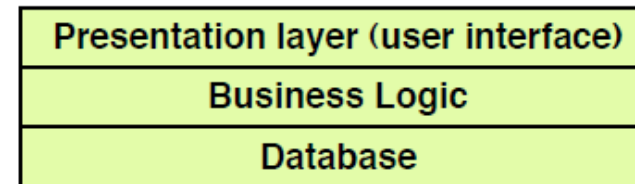
→ 2-layers: .....

- ↳ application layer
- ↳ database layer
- ↳ e.g. simple client-server model



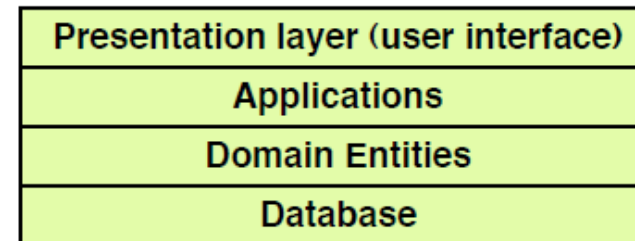
→ 3-layers: .....

- ↳ separate out the business logic
  - > helps to make both user interface and database layers modifiable



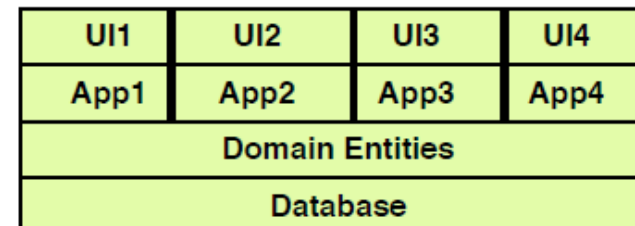
→ 4-layers: .....

- ↳ Separates applications from the domain entities that they use:
  - > boundary classes in presentation layer
  - > control classes in application layer
  - > entity classes in domain layer



→ Partitioned 4-layers .....

- ↳ identify separate applications



# Repositories

## Examples

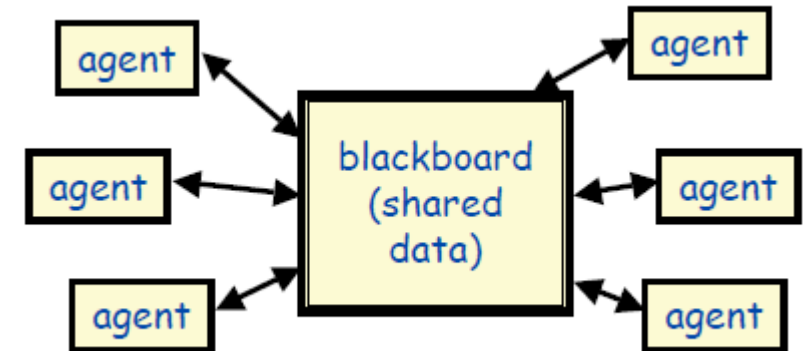
- **databases**
- **blackboard expert systems**
- **programming environments**

## ? Interesting properties

- **can choose where the locus of control is (agents, blackboard, both)**
- **reduce the need to duplicate complex data**

## ? Disadvantages

- **blackboard becomes a bottleneck**



# Repository Style Examples

---

- Information Systems
- Programming Environments
- Graphical Editors
- AI Knowledge Bases
- Reverse Engineering Systems



# Variant: Model-View-Controller

---

This pattern, also known as MVC pattern, divides an interactive application in to 3 parts as,

**model** — contains the core functionality and data

**view** — displays the information to the user (more than one view may be defined)

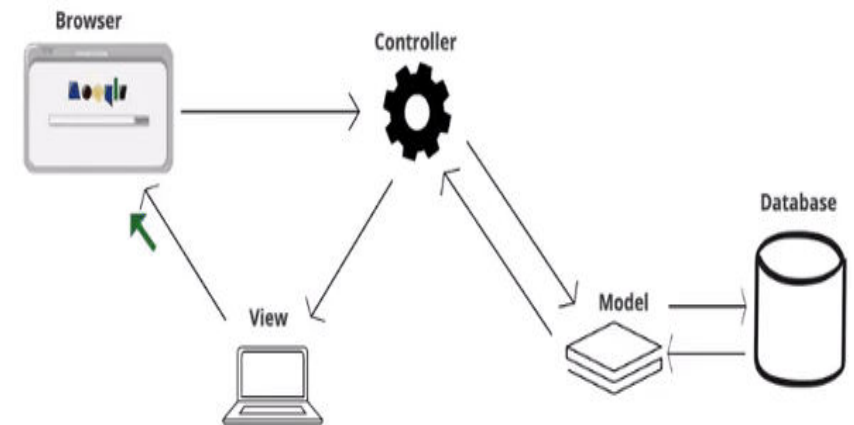
**controller** — handles the input from the user

## Usage:

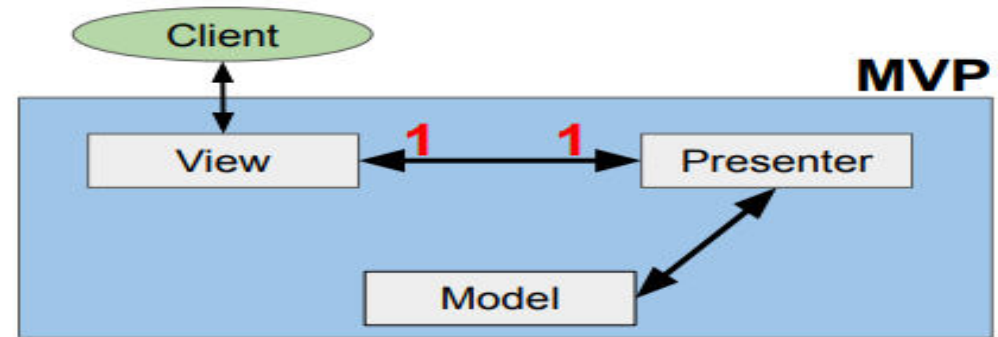
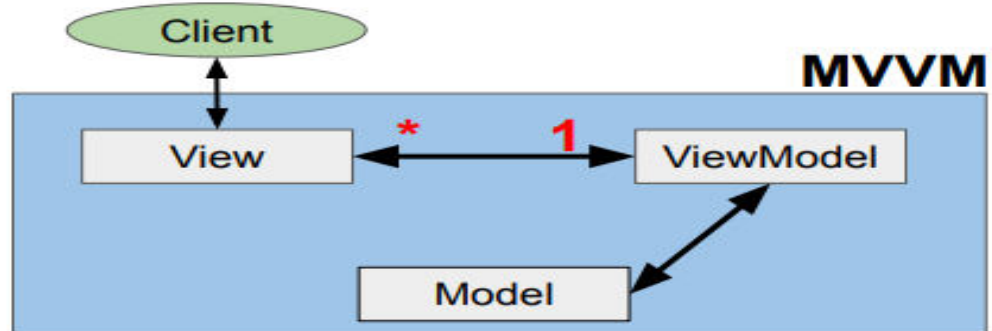
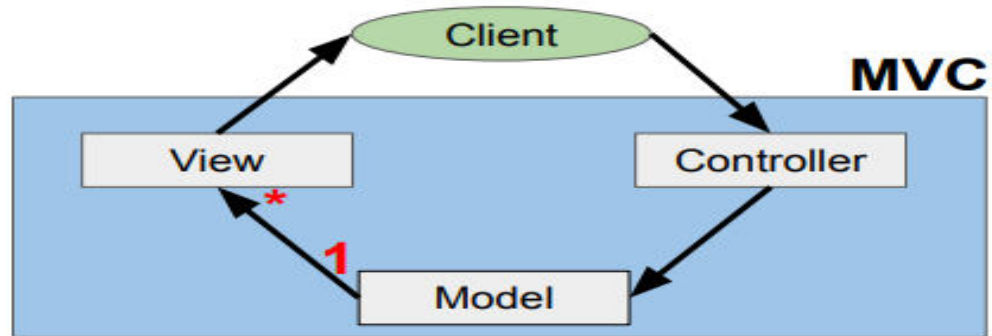
Architecture for World Wide Web applications in major programming languages.

Web frameworks such as [Django](#) and [Rails](#).

JavaSpring boot, PHP Laravel, Asp.NET MVC)



# MVC vs. MVP vs. MVVM



| Name                  | Advantages                                                                                                                                                                                                                                                                            | Disadvantages                                                                                                                                                                                                                    |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layered               | A lower layer can be used by different higher layers.<br>Layers make standardization easier as we can clearly define levels.<br>Changes can be made within the layer without affecting other layers.                                                                                  | Not universally applicable.<br>Certain layers may have to be skipped in certain situations.                                                                                                                                      |
| Client-server         | Good to model a set of services where clients can request them.                                                                                                                                                                                                                       | Requests are typically handled in separate threads on the server.<br>Inter-process communication causes overhead as different clients have different representations.                                                            |
| Master-slave          | Accuracy - The execution of a service is delegated to different slaves, with different implementations.                                                                                                                                                                               | The slaves are isolated: there is no shared state.<br>The latency in the master-slave communication can be an issue, for instance in real-time systems.<br>This pattern can only be applied to a problem that can be decomposed. |
| Pipe-filter           | Exhibits concurrent processing. When input and output consist of streams, and filters start computing when they receive data.<br>Easy to add filters. The system can be extended easily.<br>Filters are reusable. Can build different pipelines by recombining a given set of filters | Efficiency is limited by the slowest filter process.<br>Data-transformation overhead when moving from one filter to another.                                                                                                     |
| Broker                | Allows dynamic change, addition, deletion and relocation of objects, and it makes distribution transparent to the developer.                                                                                                                                                          | Requires standardization of service descriptions.                                                                                                                                                                                |
| Peer-to-peer          | Supports decentralized computing.<br>Highly robust in the failure of any given node.<br>Highly scalable in terms of resources and computing power.                                                                                                                                    | There is no guarantee about quality of service, as nodes cooperate voluntarily.<br>Security is difficult to be guaranteed.<br>Performance depends on the number of nodes.                                                        |
| Event-bus             | New publishers, subscribers and connections can be added easily.<br>Effective for highly distributed applications.                                                                                                                                                                    | Scalability may be a problem, as all messages travel through the same event bus                                                                                                                                                  |
| Model-view-controller | Makes it easy to have multiple views of the same model, which can be connected and disconnected at run-time.                                                                                                                                                                          | Increases complexity. May lead to many unnecessary updates for user actions.                                                                                                                                                     |
| Blackboard            | Easy to add new applications.<br>Extending the structure of the data space is easy.                                                                                                                                                                                                   | Modifying the structure of the data space is hard, as all applications are affected<br>May need synchronization and access control.                                                                                              |
| Interpreter           | Highly dynamic behavior is possible.<br>Good for end user programmability.<br>Enhances flexibility, because replacing an interpreted program is easy.                                                                                                                                 | Because an interpreted language is generally slower than a compiled one, performance may be an issue.                                                                                                                            |

